Searching

Subject: Searching Techniques (Sequential & Binary)

SECTION 1: CONCEPTS TO UNDERSTAND

In this section, I will explain exactly how the searching methods work using very simple English. I will describe the diagrams and steps from your slides so you can imagine the process.

1. What is Searching?

Searching is a fundamental concept in programming. You need to understand these basic ideas:

- **Definition:** Searching is a process to find if a specific element exists inside a collection of data (like a list or an array).
- **The Goal:** The goal is to determine the location of that element. If the element is found, the computer returns its **index** (its position number).
- **The Search Key:** The specific value you are looking for is called the **search key**. For example, if you are looking for "Employee ID 122", then "122" is the search key.
- **Internal vs. External Search:**
    - **Internal Search:** This is used when the amount of data is small. All the data can fit inside the computer's **RAM** (Random Access Memory).
    - **External Search:** This is used for very large amounts of data. The data is too big for the RAM. So, half the data is processed in RAM, and the other half stays in secondary storage (like the Hard Disk). *Note: Your exam focuses mostly on Internal Search using arrays.*
-

2. Sequential Search (Basic)

This is the simplest method. It is also called "Linear Search."

- **How it works:**
    - Imagine you have a row of boxes. You are looking for a red ball.
    - You open the **first** box. Is the red ball there? No.
    - You move to the **second** box. Is it there? No.
    - You move to the **third** box. Is it there? Yes!
    - You stop searching.
-
- **The Algorithm:**
    - The computer looks at array[0].
    - It compares array[0] with your search_key.
    - If they match, it stops and tells you the index (0).
    - If they do not match, it moves to array[1], then array[2], and so on.
    - **Failure:** If the computer checks every single box until the very end and never finds the key, the search fails. In your code, this usually returns -1 or false.
-

3. Sequential Search on a Sorted List

A "Sorted List" means the numbers are arranged in order (Example: 11, 22, 33, 44, 55).

- **The Problem with Unsorted Lists:** If numbers are mixed up (Example: 10, 50, 5, 20), you generally must check *every* number until you find the match or reach the end.
- **The Improvement:** If the list is sorted (ascending order), you can save time.
    - Imagine you are looking for the number **25**.
    - You check 11. (11 is less than 25, continue).
    - You check 22. (22 is less than 25, continue).
    - You check 33. **STOP.**
    - **Why stop?** Because the list is sorted. 33 is already bigger than 25. Every number after 33 will also be bigger than 25. Therefore, 25 cannot be in the rest of the list. You do not need to search further.
    - This makes the search more efficient because you terminate (end) the process earlier.
-

4. Binary Search

Binary Search is a much faster method, but it has one strict rule: **The list must strictly be sorted (ordered).**

- **The Concept (Divide and Conquer):**
    1. Instead of starting from the first box, Binary Search starts in the **MIDDLE**.
    2. It cuts the list in half with every step.
-
- **How the Algorithm Works (Step-by-Step Explanation):**
    1. **Define Range:** The computer looks at the LEFT (start) index and the RIGHT (end) index.
    2. **Find Middle:** It calculates the middle position using the formula: MIDDLE = (LEFT + RIGHT) / 2.
    3. **Compare:** It looks at the value inside array[MIDDLE].
        - **Case 1 (Match):** Is search_key == array[MIDDLE]? If yes, great! You found it. Return the index.
        - **Case 2 (Smaller):** Is search_key < array[MIDDLE]?
            - Because the key is *smaller* than the middle value, it must be in the **left side** of the list.
            - The computer ignores the right side completely.
            - It moves the RIGHT border to MIDDLE - 1.
        -
        - **Case 3 (Bigger):** Is search_key > array[MIDDLE]?
            - Because the key is *bigger*, it must be in the **right side** of the list.
            - The computer ignores the left side completely.
            - It moves the LEFT border to MIDDLE + 1.
        -
    4.
    5. **Repeat:** The process repeats with the new smaller range. It finds a new middle and compares again.
    6. **Stop:** This repeats until the value is found OR the list runs out of numbers to check (Left becomes bigger than Right).
-

5. Performance Comparison (Time Complexity)

In Computer Science, we measure how fast an algorithm is using "Big O" notation.

- **Sequential Search Efficiency:**
    - **Best Case:** O(1). This happens if the item is in the very first index (index 0). You find it immediately.
    - **Worst Case:** O(n). "n" is the total number of items. This happens if the item is at the very last position or not there at all. You have to wait to check all "n" items. It is considered slow for large data.
-
- **Binary Search Efficiency:**
    - **Best Case:** O(1). This happens if the item is exactly in the Middle on the first try.
    - **Worst Case:** O(log2 n). This is much faster.
    - **Why O(log2 n)?** Every time you make a check, you cut the search area in half (1/2). Even if you have 1,000,000 items, you can find the answer in about 20 steps.
-

SECTION 2: KEY POINTS TO MEMORIZE

These are the rules, definitions, and code steps you must write on your exam paper. Memorize these exactly.

1. Definitions

- **Search Definition:** A process to locate an element in a list and return the index of the searched element (Clifford A. Shaffer, 1997).
- **Search Key:** A specific value used to verify if an element is a member of a data set.
- **Key Field:** A group of characters or numbers used as an identifier for a record (like an ID number).
- **Internal Search:** Search implemented on a small size of data stored in **RAM**.
- **External Search:** Search implemented on a large size of data where half stays in **Secondary Storage** (hard drive) and half in RAM.

2. Rules for Algorithms

- **Sequential Search:** Can work on **Sorted** or **Unsorted** lists.
- **Binary Search:** Can **ONLY** work on a **Sorted** list (ascending or descending).
- **Efficiency Rule:**
    - Binary search is more efficient (faster) than sequential search.
    - Sequential search on sorted data is more efficient than sequential search on unsorted data.
-

3. Formulas and Operations (Crucial for Calculation Questions)

A. Binary Search Middle Calculation:
When asked to find the index to check, use this formula (integer division, round down):

$$MIDDLE=(LEFT+RIGHT)2 \text{MIDDLE} = \frac{(\text{LEFT} + \text{RIGHT})}{2} MIDDLE=2(LEFT+RIGHT)$$

B. Moving the Pointers (Binary Search in Ascending Order):

If **Key < Middle Element**: Search the Left side.

$$RIGHT=MIDDLE−1 \text{RIGHT} = \text{MIDDLE} - 1 RIGHT=MIDDLE−1$$

-

If **Key > Middle Element**: Search the Right side.

$$LEFT=MIDDLE+1 \text{LEFT} = \text{MIDDLE} + 1 LEFT=MIDDLE+1$$

-
- *(Note for Exam: If the list is sorted in **descending** order (Big to Small), swap these operators).*

4. Time Complexity Table (Efficiency)

Search Type	Best Case	Worst Case (Standard)	Description
Sequential	O(1)	O(n)	Check elements 1 by 1.
Binary	O(1)	O(log2 n)	Divide working range by 1/2 each time.

5. Steps for Sequential Search on Sorted List (Memorize for Tracing)

If asked to trace "Search Key = 25" in List: [11, 22, 33, 44]:

Check Index 0 (11).
11<2511 < 2511<25

1. . Continue.

Check Index 1 (22).
22<2522 < 2522<25

2. . Continue.

Check Index 2 (33).
33>2533 > 2533>25

3. . **STOP**.
4. **Result:** Loop breaks. Key not found. Search is terminated early to save time.

6. Important Code Logic (Syntax to remember)

A. Sequential Search Loop:

code C
downloadcontent_copy
expand_less

```
int index = -1; // -1 means record is not found initially
for (p = 0; p < array_size; p++) {
    if (search_key == array[p]) {
        index = p;  // Value found, save index
        break;      // Stop searching
    }
}
return index;
```

B. Binary Search Loop:

code C
downloadcontent_copy
expand_less

```
LEFT = 0;
RIGHT = array_size - 1;
bool found = false;

while ((LEFT <= RIGHT) && (!found)) {
    MIDDLE = (LEFT + RIGHT) / 2;

    if (array[MIDDLE] == search_key) {
        index = MIDDLE;
        found = true;
    }
    else if (array[MIDDLE] > search_key) { // Search Left
        RIGHT = MIDDLE - 1;
    }
    else { // Search Right
        LEFT = MIDDLE + 1;
    }
}
```

SECTION 3: FINAL SUMMARY

Here is the short summary connecting all the ideas for your revision.

Searching is the process of finding the position (index) of a specific item (**search key**) inside a list. The data is usually stored in an array inside the computer memory (**Internal Search**).

There are two main techniques you learned in this chapter:

1. **Sequential Search (Linear Search):**
    - This is the simplest method.
    - It checks items one by one from the start of the list to the end.
    - It works on both sorted and unsorted lists.

If the list is **unsorted**, it searches typically
$O(n)O(n)O(n)$

    - time (slow).
    - If the list is **sorted**, the code can "break" (stop) early if it encounters a number bigger than the search key, making it slightly more efficient.
2.
3. **Binary Search:**
    - This is a sophisticated, efficient method.
    - **Crucial Condition:** The list **must** be sorted before you use this.
    - It uses a "Divide and Conquer" strategy. It calculates a MIDDLE index.
    - Based on the comparison with the middle value, it throws away half of the list (either the left half or the right half).
    - It repeats this until the value is found.

The time complexity is
$O(log2n)O(\log_2 n)O(log2n)$

, which means the search area size reduces from
nnn

to
$n/2n/2n/2$

to
$n/4n/4n/4$

    - with every step.
4.

Conclusion: For small sets of data, Sequential search is okay. For large sets of data, Binary search is much faster and better, but you must sort the data first. Sequential search on unsorted data is the slowest method.

Linked List

Subject: Chapter 7 - Linked List

SECTION 1: CONCEPTS TO UNDERSTAND

In this section, I will explain the "story" behind Linked Lists. You must understand *why* we use them and *how* they are different from Arrays.

1. Linear Lists (The Big Picture)

A **List** is just a group of objects organized in a specific order (sequence).
There are two main categories of lists mentioned in your slides:

- **Linear List:** Data is arranged in a straight line, one after another. Examples: **Array**, **Linked List**, **Stack**, and **Queue**.
- **Non-Linear List:** Data is not in a simple sequence. Examples: **Tree** and **Graph**.

Important Difference:

- **Array & Linked List:** You can insert or delete data anywhere (front, middle, back). There are no strict rules.
- **Stack & Queue:** These have strict rules. For a **Stack**, you can only touch the top. For a **Queue**, you enter at the back and leave from the front.

2. The Problem with Arrays (Why do we need Linked Lists?)

You already know what an Array is. It is a linear list where data is stored **side-by-side** in memory (contiguous).

The Good Part of Arrays:

- **Fast Access:** You can jump directly to any item if you know the index (Example: Student[3]). This is called **Random Access**.

The Bad Part (The Drawbacks) of Arrays:

1. **Fixed Size:** When you create an array, you must decide the size immediately (e.g., int array[10]).
    - If you only use 3 spaces, you **waste space**.
    - If you need 11 spaces, you cannot add more. The array is full.
2.
3. **Slow Insertion (Adding data):**
    - Imagine a bench where people are sitting tightly.
    - If you want to sit in the **middle** (index 3), everyone after index 3 must stand up and **shift** (move) one step to the right to make space for you.
    - The computer has to move half the list just to add one number. This is slow.

4. **Slow Deletion (Removing data):**
    - If a person in the middle leaves, you have an empty hole.
    - To fix the hole, everyone must **shift** to the left to close the gap.
5.

3. The Solution: Linked List

A **Linked List** solves the problems of the Array.

- It does **not** store data side-by-side.
- It uses a "treasure hunt" style. Each item holds a map to find the next item.

What is a Node?
A Linked List is built from **Nodes**. You must understand the structure of a Node.
A Node has two parts inside it:

1. **Data:** The actual information (like a number 4.5 or a name Ahmad).
2. **Pointer (Next):** This points to the memory address of the **next** node.

Special Pointers:

- **Head:** This is a special pointer that shows where the list starts. It points to the first node.
- **NULL:** The last node in the list points to NULL. This means "The End." There is no node after this one.

4. Variations of Linked Lists

There are different ways to connect the nodes:

- **Singly Linked List:** Moves in one direction only.
    - (Node A) points to (Node B). You cannot go back from B to A.
-
- **Doubly Linked List:** Moves in two directions (Forward and Backward).
    - Each node has **two pointers**: Next (successor) and Previous (predecessor).
    - Easier to travel backwards, but uses more memory for the extra pointer.
-
- **Circular Linked List:** The **Last Node** points back to the **First Node** (Head). It is a circle. There is no NULL.
- **Sorted vs. Unsorted:**
    - **Sorted:** You must insert numbers in the correct order (Small to Big).
    - **Unsorted:** You can add numbers anywhere.
-

5. Implementing the Logic (Operations)

The slides provide C++ logic for managing these lists.

- **Classes Used:**
  - Class Node: Contains the data and the next pointer.
  - Class List: Contains the head pointer and all the functions (Insert, Find, Delete).
-
- **How Insertion Works (Algorithm):**
  - *Goal:* Add a new node (X) but keep the list **Sorted**.
  - *Step 1:* Find the correct spot. Look at the list until you find a number bigger than X. You need to stop right before it.
  - *Step 2:* Use two pointers, currNode (Current) and prevNode (Previous), to travel the list.
  - *Step 3:* Create the new node.
  - *Step 4 (Linking):* Make the **new node** point to the **current node**. Then, make the **previous node** point to the **new node**. Now, the chain is connected.
-
- **How Deletion Works (Algorithm):**
  - *Goal:* Remove a node with value X.
  - *Step 1:* Use currNode and prevNode to search for X.
  - *Step 2:* If found, you must unlink it.
  - *Step 3:* Connect prevNode->next to currNode->next. This "skips" over the node you want to delete.
  - *Step 4:* Use delete currNode to remove it from memory completely.
-

SECTION 2: KEY POINTS TO MEMORIZE

These are the definitions, differences, and code concepts you must remember for the exam paper.

1. Important Definitions

- **List:** A group of objects organized in sequence.
- **Linear List:** A list where data is organized in a sequence (e.g., Array, Linked List).
- **Linked List:** A series of connected nodes where each node contains **data** and a **pointer** to the successor (next node).
- **Head:** A pointer variable that points to the **first node** in the list.
- **Successor:** The node that comes *after* the current node.
- **Predecessor:** The node that comes *before* the current node (used in Doubly Linked Lists).

2. Differences: Array vs. Linked List (Crucial for Exam)

Feature	Array	Linked List
Size	Fixed at declaration (Static).	Dynamic (Can grow and shrink).
Memory	Contiguous (Side-by-side).	Not contiguous (Scattered).
Data Access	Random Access (Fast) using index [i].	Linear Access (Slow). Must visit nodes one by one.
Insertion	Slow. Must shift elements down to make room.	Fast. Just change pointers. No shifting.
Deletion	Slow. Must shift elements up to close gap.	Fast. Just change pointers and delete node.
Storage	Wastes space if not full.	Uses extra space for pointers in every node.

3. Structure Declaration (Memorize Syntax)

You must know how to write a Node in C++ or Struct.

code C++
downloadcontent_copy
expand_less

```
class Node {
   public:
      double data;    // The value
      Node* next;     // The pointer to the next node
};
```

You must know the List class basics:

code C++
downloadcontent_copy
expand_less

```
class List {
   private:
      Node* head;     // Points to start
   public:
      List() { head = NULL; } // Constructor sets head to NULL
      bool IsEmpty() { return head == NULL; }
      // Operations: InsertNode, DeleteNode, FindNode, DisplayList
};
```

4. Algorithm Steps (Step-by-Step for Logic Questions)

A. Inserting a Node (Sorted Order):

1. **Allocate** memory for the new node (newNode).
2. **Assign** value to newNode->data.
3. **Traverse** (Search) to find the correct location using prevNode and currNode.
4. **Link**:
    - newNode->next = currNode; (Connect new node to the rest of the list).
    - prevNode->next = newNode; (Connect the previous list to the new node).
    - *Special Case:* If inserting at the very beginning (Index 0), update head = newNode.
5.

B. Deleting a Node:

1. **Traverse** to find the node where currNode->data == x.
2. **Unlink**:
    - prevNode->next = currNode->next; (The previous node points to the node *after* the current one).
3.
4. **Delete**:
    - delete currNode; (Release memory).
    - *Special Case:* If deleting the first node, move head: head = head->next.
5.

C. Destructor (~List()):

- The destructor must look at every node one by one and delete them to free the memory. You cannot just delete head; you must walk through the list deleting nodes individually.

5. Loop Conditions to Remember

- **To Traverse until the end:**
  while (currNode != NULL) { ... }
- **To find a sorted position (Insertion):**
  while (currNode != NULL && x > currNode->data) { ... }
  (Stop if you reach the end OR if you find a value bigger than X).
- **To find a specific value (Find/Delete):**
  while (currNode != NULL && currNode->data != x) { ... }

SECTION 3: FINAL SUMMARY

To summarize Chapter 7:

A **Linked List** is a dynamic way to store data. Unlike an **Array**, which is like a fixed row of lockers, a Linked List is like a scavenger hunt where every item has a pointer (map) to the next item.

Why use Linked Lists?
Because Arrays have a **fixed size** and are **slow** when you want to insert or delete items in the middle (because of shifting data). Linked Lists allow you to add and remove items easily just by changing the pointers.

What is the Trade-off?
While insertion and deletion are fast, **accessing** data is slow. In an array, you can check Array[5] instantly. In a Linked List, you must start at the **Head** and travel Next -> Next -> Next -> Next -> Next to find the 5th item.

You implement a linked list using a **Node** class (which holds data and a *next pointer). You must memorize how to **Insert**, **Delete**, and **Find** nodes by carefully managing the Head, currNode (Current), and prevNode (Previous) pointers. If you do not handle the pointers correctly (especially NULL), the program will crash or lose the list.

Stack

Subject: Chapter 8 - Stack

SECTION 1: CONCEPTS TO UNDERSTAND

In this section, I will explain what a Stack is, how it works in code, and how we use it to solve math problems.

1. What is a Stack? (The Basic Idea)

Imagine a stack of heavy books or a stack of plates in a cafeteria.

- You put a new plate **on top**.
- When you need a plate, you take the one **from the top**.
- You cannot take a plate from the bottom without knocking everything over.

This leads to the most important rule of this chapter:
LIFO: Last In, First Out.
The **Last** item you put in is the **First** item that comes out.

Real Life Examples:

1. **Narrow Driveway:** Imagine a driveway where cars park one behind the other. A BMW goes in first. A Lexus goes in second. A Benz goes in last. To get the BMW out, the Benz must leave first, then the Lexus.
2. **Books/Plates:** As explained above.

Computer Examples:

1. **Undo Button:** When you type in Word or Excel and make a mistake, you press "Undo". It removes the *last* thing you did.
2. **Web Browser History:** The "Back" button takes you to the *last* page you visited.

2. Main Operations (The Actions)

You can do specific things with a Stack. In the exam, you cannot just say "add" or "delete". You must use the correct technical names:

- **Push:** Adding a new item to the **top** of the stack.
- **Pop:** Removing/Deleting the item from the **top** of the stack.
- **StackTop (or Peek):** Looking at the item on top *without* removing it. You just check what the value is.
- **CreateStack:** Making a new, empty stack.
- **IsEmpty:** Checking if the stack has zero items.
- **IsFull:** Checking if the stack has no more space (this is only for Arrays).

3. Implementing a Stack: Array vs. Linked List

Just like Chapter 7, we can build a Stack in two ways.

A. Array Implementation (The Box Method)

- You create an array (a fixed list).
- You need a variable called top.
- top is an integer (a number). It tells you the **index** of the top item.
- **Empty:** If the stack is empty, top = -1.
- **Push:** To add an item, you move top up (top + 1) and put the data there.
- **Problem:** An array has a fixed size (e.g., max_size = 100). If you try to push the 101st item, the stack is **Full** and you cannot add it.

B. Linked List Implementation (The Chain Method)

- You use pointers.
- You do **not** need a variable called top (index). Instead, the head pointer acts as the top.
- **Push:** You create a newnode. This new node becomes the new head (the new top). The old head becomes the second node.
- **Pop:** You delete the first node (head) and make head point to the next one.
- **Benefit:** A Linked List is flexible. It never gets "Full" unless the computer memory runs out. You do not need the isFull() function.

4. Applications of Stack (What is it used for?)

Stacks are not just for storing data; they are algorithms to solve complex problems.

Application 1: Checking Balanced Parentheses ()

- Computers need to check if code is written correctly. Every opening bracket (must have a closing bracket).
- **Logic:**
    - Read the text from left to right.
    - If you see (, **Push** it onto the stack.
    - If you see), **Pop** one (from the stack.
    - **Result:** If the stack is empty at the end, it is balanced. If the stack is empty but you still see a), or if the text ends and the stack still has (, it is **not balanced**.
-

Application 2: Algebraic Expressions
We usually write math like A + B. This is called **Infix**.
Computers prefer different ways to read math so they don't get confused by parentheses or order of operations.

- **Infix:** Operator is *between* operands. (e.g., A + B)
- **Prefix:** Operator is *before* operands. (e.g., + A B)
- **Postfix:** Operator is *after* operands. (e.g., A B +)

Application 3: Evaluating Expressions
How does the computer solve 2 7 * 18 - 6 + (which is a postfix equation)?

1. It reads 2. It is a number. **Push** to Stack.
2. It reads 7. It is a number. **Push** to Stack.
3. It reads * (Multiply). It **Pops** the 7 and 2. It calculates 2 * 7 = 14. **Push** 14.
4. It reads 18. **Push**.
5. It reads - (Minus). It **Pops** 18 and 14. It calculates 14 - 18 = -4. **Push** -4.
   (And so on...)

SECTION 2: KEY POINTS TO MEMORIZE

These are the definitions, rules, and code snippets you must memorize for the final exam.

1. Fundamental Definition

- **Stack:** A linear list where all additions and deletions are restricted to one end, called the **top**.
- **Concept: LIFO** (Last In, First Out). The last one pushed is the first one popped.

2. Array Implementation Rules

You must remember how top changes in an array.

- **Variables needed:** An array (e.g., data[100]) and an integer int top.
- **Initialization:** top = -1; (Stack is empty).
- **IsFull:** Returns true if top == size - 1.
- **IsEmpty:** Returns true if top == -1.

Formulas for Array Operations:

- **Push (Add):**
  1. top = top + 1; (Increase index)
  2. data[top] = newItem; (Store data)
-
- **Pop (Delete):**
  1. item = data[top]; (Save value to return it)
  2. top = top - 1; (Decrease index)
-

3. Linked List Implementation Rules

- **Variables needed:** A class/struct Node with data and *next pointer. A pointer *head (which acts as the Top).
- **Initialization:** head = NULL;.
- **IsFull: NOT NEEDED** for Linked List stacks.
- **IsEmpty:** Returns true if head == NULL.

Code Logic for Linked List Operations:

Push (Insert at Front):

```
 code C++
downloadcontent_copy
expand_less
nodeStack *newnode = new nodeStack;
newnode->data = newitem;
newnode->next = head; // Point new node to old head
```

- head = newnode; // Update head to be the new node

Pop (Delete from Front):

```
 code C++
downloadcontent_copy
expand_less
nodeStack *delnode = head; // Point to the top node
head = head->next;  // Move head to the second node
```

- delete delnode; // Delete the old top

4. Algebraic Conversions (Important Rules)

You must know how to convert **Infix** to **Postfix** manually using a Stack.

Precedence Rules (Who is stronger?):

1. ( ) (Strongest - Force calculation)
2. * / (Medium Strength - done before addition)
3. + - (Weakest Strength - done last)

Algorithm for Converting Infix to Postfix:

1. If you see an **Operand (A, B, 1, 2):** Send it directly to the **Output/Postfix string**.
2. If you see an **Operator (+, -, , /):*
    - Compare it with the operator currently inside the Stack.
    - If the new operator is **weaker or equal** to the stack top, **POP** the stack operator first, then **PUSH** the new one.
    - If the new operator is **stronger**, simply **PUSH** it on top.
3.
4. If you see (: Push it.
5. If you see): Pop everything until you find the matching (.

5. Evaluating Postfix Expressions

Rule:
When calculating: Operand2 (Operator) Operand1.

- **Example:** If stack has Top -> 7 and below it is 18, and operation is -:
    - Pop 7 (This is Opn1).
    - Pop 18 (This is Opn2).
    - Calculate: 18 - 7. (Not 7 - 18).
    - **Memorize this:** result = opn2 operator opn1.
-

SECTION 3: FINAL SUMMARY

Here is a short summary to combine everything.

Chapter 8 teaches you about the **Stack**, which is a Data Structure that follows the **LIFO (Last In, First Out)** rule. Imagine a narrow driveway where the last car in must be the first car out.

There are **two main ways** to write code for a Stack:

1. **Array:** Simple but fixed size. You use an integer top index. Empty is -1. Push is top++. Pop is top--. It needs an isFull() check.
2. **Linked List:** Dynamic and flexible size. You use a pointer *head (or *top). New items are added to the front (push) and removed from the front (pop). It uses pointers (next) to connect nodes.

Main Operations:

- **Push:** Add to top.
- **Pop:** Remove from top.
- **StackTop:** Look at the top value without removing.
- **IsEmpty:** Check if empty.

Applications:
The stack is critical for algorithms like **"Undo" buttons** and checking **Balanced Parentheses**. It is widely used in math to convert **Infix** (A+B) into **Postfix** (AB+), which helps computers calculate math equations easily using the evaluation algorithm (Opn2 Operator Opn1).

Queue

Subject: Chapter 9 - Queue (Array & Linked List)

SECTION 1: CONCEPTS TO UNDERSTAND

In this section, I will explain the "story" of the Queue. You must understand *how* items move through it and *why* different methods are used to build it.

1. What is a Queue? (The Core Concept)

The most important rule for a Queue is **FIFO**.

- **F.I.F.O.** stands for **First-In, First-Out**.
- This is the exact opposite of a Stack (which is Last-In, First-Out).

Real Life Example:
Think of a **cashier line** in a store or a student queue at a cafeteria.

- The first person who stands in line is the first person to pay and leave.
- New people join the line at the **Back** (Rear).
- People leave the line from the **Front**.
- Middle people cannot leave; they must wait for their turn.

Computer Applications:

- **Printer Jobs:** If you send three documents to print, the printer prints the first one first. The others wait in a queue.
- **Keyboard Buffer:** The keys you press are processed in the order you pressed them.

2. Structure of a Queue

A queue always has two main position markers:

- **Front (Head):** This points to the item that has been waiting the longest. This is where we remove items.
- **Back (Rear):** This points to the item that was added most recently. This is where we add new items.

3. The Main Operations (Actions)

You must understand what these function names mean:

- **enQueue(item):** "Insert". This adds a new item to the **back** of the queue.
- **deQueue():** "Remove". This removes the item from the **front** of the queue.
- **getFront():** "Peek". This looks at the first item without removing it.
- **getRear():** Look at the last item.
- **isEmpty():** Check if the queue has zero items.
- **isFull():** Check if the queue has no space left (Used mainly for Arrays).

4. Implementation Type A: Linear Array

We can build a queue using a standard array (a fixed list of boxes).

- We use two integer variables: front and back to track positions.
- **Problem - "Rightward Drifting":**
    - Imagine an array of 5 boxes.
    - You fill boxes 0, 1, 2. (Back is at 2).
    - You delete boxes 0 and 1. (Front moves to 2).
    - The Front keeps moving to the right. The Back keeps moving to the right.
    - Eventually, the **Back** hits the end of the array. Even though you deleted items from the front and have empty space on the left, the computer thinks the queue is full.
    - This is called "Drifting".
    - **Solution?** Shifting all items back to index 0 is possible, but it is **slow** and makes the computer work too hard. The better solution is a Circular Array.
-

5. Implementation Type B: Circular Array

This solves the "Rightward Drifting" problem.

- Imagine the array is not a straight line, but a circle (like a clock).
- When the back reaches the end of the array, the next step wraps around to the beginning (index 0).
- **Modulo Arithmetic (%):** This is a special math operator. We use it to calculate the position in the circle. It resets the number to 0 when it hits the maximum size.

6. Implementation Type C: Linked List

This uses **Nodes** and **Pointers** instead of a fixed box.

- **Why is it better?** It is dynamic. It never gets "Full" (unless the computer runs out of RAM).
- **Pointers Needed:**
    - **Linear Linked List:** Needs two pointers. One pointing to Front and one pointing to Back.
    - **Circular Linked List:** Needs only **one** pointer (Back).
        - *Concept:* In a circle, the node *after* the Back is automatically the Front. So if you know where the Back is, you also know where the Front is. This saves memory.
    -
-

SECTION 2: KEY POINTS TO MEMORIZE

These are the definitions, formulas, and rules you must write exactly on the final exam.

1. Important Properties

- **Property Name:** FIFO (First-In, First-Out).
- **Insertion Location:** Always at the **Rear** (Back).
- **Deletion Location:** Always at the **Front**.

2. ADT Operations (Key definitions)

- **createQueue():** Initialize queue. For Linear Array: front = 0, back = -1. For Linked List: front = NULL, back = NULL.
- **isEmpty():** Returns true if the queue has no elements.
- **isFull():** (Array only) Returns true if count == MAX_QUEUE.
- **destroyQueue():** Deletes all elements from memory (Crucial for Linked Lists to prevent memory leaks).

3. The "Circular Array" Formula (Crucial for Calculation)

In a circular implementation, you do not simply do back + 1. You must memorize this formula to calculate the next position:

Back=(Back+1)%MAX_QUEUE \text{Back} = (\text{Back} + 1) \% \text{MAX_QUEUE}
Back=(Back+1)%MAX_QUEUE

- **Logic:** If MAX_QUEUE is 5, and you are at index 4:
    - (4 + 1) % 5 = 0.
    - The index resets to 0 correctly.
-

4. Code Logic to Memorize (Linked List Queue)

A. Structure of a Node:

code C++
downloadcontent_copy
expand_less

```
struct nodeQ {
   char item;       // The Data
   nodeQ *next;     // Pointer to next node
};
```

B. Algorithm: EnQueue (Insert at Back):

1. Create a newPtr node.
2. If the queue is **Empty**:
    - frontPtr = newPtr;
    - backPtr = newPtr;
3.
4. If the queue is **Not Empty**:
    - backPtr->next = newPtr; (Link old back to new node).
    - backPtr = newPtr; (Update back pointer).
5.

C. Algorithm: DeQueue (Delete from Front):

1. Check if isEmpty(). If yes, error.
2. If **Not Empty**:
    - Save current front: tempPtr = frontPtr;
    - Move front forward: frontPtr = frontPtr->next;
    - Delete old node: delete tempPtr;
    - **Special Case:** If the queue had only 1 item, after deletion, set backPtr = NULL.
3.

5. Comparison Table (Array vs. Linked List)

Feature	Array Implementation	Linked List Implementation
Size	Fixed (Static). Limited by definition.	Dynamic. Grows as needed.
Fullness	Must check isFull(). Can overflow.	Theoretically never full.
Complexity	Simple, but has "Drifting" issues.	Slightly more complex code (pointers).
Overhead	None (direct access).	Pointer manipulation overhead.
Circular	Requires % operator.	Circular Link means Back->Next == Front.

6. Distinguishing Full vs. Empty in Circular Array

When using a circular array, the pointers for "Full" and "Empty" can look the same.

- **Problem:** If front and back are close, it is hard to tell.
- **Solution:** Use a variable called count.
    - Initialize count = 0.
    - When adding (enQueue): count++.
    - When removing (deQueue): count--.
    - **Empty:** count == 0.
    - **Full:** count == MAX_QUEUE.
-

SECTION 3: FINAL SUMMARY

To simplify everything for your exam revision:

A **Queue** is a data structure used for simulations, printers, and buffering. It strictly follows **FIFO** (First-In, First-Out).

There are three main ways to build it:

1. **Linear Array:** Easy to make, but has a **Rightward Drifting** problem. You keep deleting from the front, leaving wasted empty space. We avoid this method in efficient software.
2. **Circular Array:** The Best Array method. It treats the end of the array as if it is connected to the start. You must memorize the modulo formula: (index + 1) % size. You also track a count variable to distinguish between a full queue and an empty one.
3. **Linked List:** The most flexible method. You generally maintain two pointers: frontPtr and backPtr.
    - **Insertion (enQueue)** happens at the **Back**.
    - **Deletion (deQueue)** happens at the **Front**.
    - A **Circular Linked List** is smart because you only need one pointer (Back). The node after Back (Back->Next) is always the Front.
4.

Exam Tip: If asked to insert items into a Linear Array Queue, watch out for the array size limit. If asked to insert into a Circular Queue, remember to "wrap around" to index 0 when you reach the limit.

Tree

Subject: Chapter 10 - Trees & Binary Search Trees (BST)

SECTION 1: CONCEPTS TO UNDERSTAND

In this section, I will explain the "story" of Trees. We start with the general idea and then focus on the most important specific type: the Binary Search Tree.

1. What is a General Tree? (The Basic Structure)

A **Tree** is a way to organize data in a hierarchy (like a family tree or an organizational chart in a company).

- It is **Non-Linear**. (Unlike Arrays, Linked Lists, Stacks, and Queues which are straight lines).
- It is a collection of **Nodes** connected by **Edges** (lines).

Analogy:
Think of a company.

- The **CEO** is at the very top.
- The Managers report to the CEO.
- The Workers report to the Managers.
  This shape is a Tree.

2. Tree Terminology (The Vocabulary)

You **must** know these words to answer exam questions involving diagrams.

- **Root:** The single top node. It has no parent. (Example: The CEO).
- **Parent:** A node that has nodes below it.
- **Child:** A node connected to a parent above it.
- **Siblings:** Nodes that share the same parent (like brothers and sisters).
- **Leaf:** A node at the very bottom that has **no children**.
- **Ancestor:** Any node sitting above a specific node on the same path (Parent, Grandparent, etc.).
- **Descendant:** Any node sitting below a specific node (Child, Grandchild, etc.).
- **Subtree:** If you cut off a branch of a tree, that branch itself looks like a smaller tree.
- **Depth:** The distance from the Root to a node (Level 0, Level 1, Level 2...).
- **Height:** The length of the longest path from the root to the deepest leaf.

3. What is a Binary Tree?

A **General Tree** can have unlimited children (a parent can have 100 kids).
A **Binary Tree** has a strict rule:

- Every node can have **at most TWO children**.
- We call them the **Left Child** and the **Right Child**.

Special Types of Binary Trees:

- **Full Binary Tree:** Every node has either **0** or **2** children. No node has just 1 child.
- **Complete Binary Tree:** All levels are completely full, except maybe the last level, which is filled from **left to right**.
- **Balanced Binary Tree:** The height of the left side and the right side are almost the same (difference is not more than 1).

4. What is a Binary Search Tree (BST)? (Crucial for Exam)

This is a specific version of a Binary Tree designed for **fast searching**. It follows a specific math rule:

The BST Rule:
For any Node N:

- All values in the **Left Subtree** must be **SMALLER** than N.
- All values in the **Right Subtree** must be **BIGGER** than N.

Example:

- If Root is 50.
- The Left Child can be 40. (40 < 50)
- The Right Child can be 60. (60 > 50)
- This rule repeats for every single node in the tree.

5. Tree Traversals (How to read the tree)

Traversal means "visiting every node in the tree". Because trees aren't straight lines, there are three ways to walk through them.

1. **Pre-Order (Root First):**

Order: Root
→\rightarrow→

Left
→\rightarrow→

- Right
- *Usage:* Useful for copying trees.

2.
3. **In-Order (Sorted):**

Order: Left
→\rightarrow→

Root
→\rightarrow→

- Right
- *Usage:* **Important:** This prints the numbers in **Sorted Order** (Small to Big).

4.
5. **Post-Order (Root Last):**

Order: Left
→\rightarrow→

Right
→\rightarrow→

- Root
- *Usage:* Useful for deleting trees.

6.

SECTION 2: KEY POINTS TO MEMORIZE

These are the definitions, rules, and formulas you need for exam questions.

1. Terminology Definitions

- **Node:** An item stored in the tree containing data and pointers.
- **Root:** The unique starting node.
- **Height of Tree:** The number of nodes along the longest path from root to leaf.

Formula for Max Height:

nnn

- (if the tree is a straight line).

Formula for Min Height:

log2(n+1)\log_2(n+1)log2(n+1)

- (if the tree is full).

2. The Rules of BST Operations

A. Insertion (Adding a Number):

1. Start at the Root.
2. Compare the new number with the current node.
3. If **New < Current**: Go **Left**.
4. If **New > Current**: Go **Right**.
5. Repeat until you hit an empty spot (NULL). Put the new node there.

B. Searching:

1. Start at Root.
2. If **SearchKey == Root**: Found it!
3. If **SearchKey < Root**: Search the **Left** subtree.
4. If **SearchKey > Root**: Search the **Right** subtree.

C. Deletion (Removing a Node) - 3 Cases:

1. **Delete a Leaf:** Just remove it (Set parent pointer to NULL).
2. **Delete Node with 1 Child:** The parent points directly to the child (Bypass the deleted node).
3. **Delete Node with 2 Children:**
    - Find the **Minimum** value in the **Right Subtree** (this is called the *Inorder Successor*).
    - Replace the node you want to delete with that value.
    - Delete the duplicate node found in the right subtree.
4.

3. Traversal Patterns (Memorize These!)

Given this small tree:

code Code
downloadcontent_copy
expand_less

```
A
  / \
 B   C
```

- **Pre-Order:** A, B, C (Root, Left, Right)
- **In-Order:** B, A, C (Left, Root, Right)
- **Post-Order:** B, C, A (Left, Right, Root)

4. Efficiency (Time Complexity)

You must memorize how fast a Binary Search Tree works compared to a Linked List.

Operation	Average Case (Balanced)	Worst Case (Straight Line)
Search	O(logn)O(\log n)O(logn)	O(n)O(n)O(n)
Insert	O(logn)O(\log n)O(logn)	O(n)O(n)O(n)
Delete	O(logn)O(\log n)O(logn)	O(n)O(n)O(n)

5. Code Structure (Syntax)

How to write a Tree Node in C++:

code C++
downloadcontent_copy
expand_less

```
struct TreeNode {
   int info;          // The Data
   TreeNode *left;     // Pointer to left child
   TreeNode *right;    // Pointer to right child
};
```

SECTION 3: FINAL SUMMARY

Trees are hierarchical data structures. The top is the **Root** and the bottoms are **Leaves**.

A **Binary Tree** allows each node to have maximum 2 children.

A **Binary Search Tree (BST)** is a special Binary Tree used for organizing numbers efficiently.

- **Rule:** Everything on the **Left is Smaller**. Everything on the **Right is Bigger**.
- Because of this rule, searching is very fast (like Binary Search on an array).

You perform 3 types of movement (Traversal):

1. **Pre-Order:** Root first.
2. **In-Order:** Root in the middle (Produces a Sorted list: 1, 2, 3...).
3. **Post-Order:** Root last.

Operations:

- **Insertion** is easy: Just follow the Less/Greater rule until you find an empty spot.
- **Deletion** is harder: If the node has 2 children, you must swap it with the smallest node from the right side.

Exam Tip: Practice drawing the tree. If the question gives you a list of numbers "5, 3, 7, 2, 4" and asks you to insert them, draw 5 as the root, then check 3 (left of 5), then check 7 (right of 5), then 2 (left of 3), etc.

Final Exam prepartion

THE ULTIMATE DATA STRUCTURES & ALGORITHMS SURVIVAL GUIDE

How to Use This Guide:

1. **Do not panic.** This subject is visual. If you can draw it, you can pass.
2. **Read Section by Section.** Do not skip.
3. **Memorize the "Golden Rules".** These are the answers to the hardest questions.

TOPIC 1: SEARCHING TECHNIQUES

(Likelihood in Exam: 100% - Usually in Part A or Part B Question 1)

You only need to know two types of searching. The exam asks you to **calculate the steps** or **show the tracing**.

1. Sequential Search (Linear Search)

- **What it is:** Checking items one by one. Like looking for a specific shirt in a pile of clothes.
- **Best Case:** O(1). (You find it instantly at the start).
- **Worst Case:** O(n). (It is at the very end, or not there at all).
- **Prerequisite:** The list can be unsorted (messy) or sorted.

Exam Trick:
If the list is **Sorted** (e.g., 2, 4, 6, 8, 10) and you are looking for **5**:

1. Check 2 (Too small, continue).
2. Check 4 (Too small, continue).
3. Check 6 (Too big!). **STOP.**
   Reason: Because the list is sorted, 5 cannot be after 6. You stop early. The exam often asks "How many comparisons?" In this case, it is **3 comparisons**.

2. Binary Search (The Favorite Exam Question)

- **Crucial Rule:** The list **MUST** be sorted. You cannot use Binary Search on a messy list.
- **Logic:** Divide and Conquer. You jump to the middle. If the key is smaller, you ignore the right side. If bigger, ignore the left.

How to Solve the Binary Search Table Question:

From Past Exams (2013, 2024): You are given an array and asked to fill a table with LEFT, RIGHT, and MID.

Memorize the Variables:

- **LEFT (First Index):** Usually 0.
- **RIGHT (Last Index):** Size - 1.
- **MID (Formula):** (LEFT + RIGHT) / 2. (Integer math: round down).

Example Trace (Search 20):
Array: [10, 15, 20, 25, 30] (Indexes 0 to 4).
Goal: Find **20**.

Step 1:

- LEFT = 0
- RIGHT = 4
- MID = (0 + 4) / 2 = **2**.
- Value at MID (index 2) is **20**.
- Does 20 == 20? Yes! **Found.**

Example Trace (Search 25 - if 20 was not the target):

- If we looked for 25:
- Value 20 < 25. So we must look at the **Right** side.
- **New LEFT** = MID + 1. (Index 3).
- **New RIGHT** = Old RIGHT. (Index 4).
- **New MID** = (3 + 4) / 2 = 3.5 -> **3**.

Golden Rule for Searching

- **Sequential Search** is slow O(n) but works on messy data.
- **Binary Search** is fast O(log n) but requires sorted data.

TOPIC 2: LINKED LISTS

(Likelihood in Exam: 100% - Coding and Diagram Drawing)

This is the hardest topic for many, but I will make it simple.

The Concept

An Array is like a long bench. Everyone sits side-by-side.
A Linked List is like a treasure hunt.

- You have a **Node** (The paper).
- The Node has **Data** (The clue).
- The Node has a **Pointer/Link** (Direction to the next paper).

How to Draw It (Important for Part B questions):

Nodes are boxes with two sections.
[Data | Pointer] --> [Data | Pointer] --> NULL

- **Head:** A special pointer that points to the first box.
- **NULL:** The last box points to nothing.

Types of Linked Lists (Asked in Objectives):

1. **Singly Linked List:** A -> B -> C. You can only move forward.
2. **Doubly Linked List:** A <-> B <-> C. You can go forward and backward. Each node has prev and next pointers.
3. **Circular Linked List:** A -> B -> C -> A. The last node points back to the Head. No NULL.

Exam Code Logic (You must write this):

1. Create a Node Code:

code C++
downloadcontent_copy
expand_less

```
struct Node {
    int data;       // The information
    Node *next;     // The pointer to the next node
};
```

2. Inserting a New Node (e.g., at the Front):
Goal: Put "NewNode" before "Head".

1. Create NewNode.
2. NewNode->next = head; (Connect new guy to the old leader).
3. head = NewNode; (New guy is now the leader).

3. Inserting a Node (e.g., in the Middle/End):
Goal: Insert node 'P' between 'A' and 'B'.
Code logic often tested:

code C++
downloadcontent_copy
expand_less
```
P->next = A->next;  // 1. Connect P to B
A->next = P;        // 2. Connect A to P
```

Warning: Never break the chain before connecting the new node, or you lose the rest of the list!

4. Deleting a Node:
Goal: Remove node 'Target'.
You need a previous pointer.

1. previous->next = Target->next; (Skip over the target).
2. delete Target; (Throw away the target).

Visualizing the Loop Code (Past Exam Question):

If you see code like this:

code C++
downloadcontent_copy
expand_less
```
Node *temp = head;
while (temp != NULL) {
   cout << temp->data;
   temp = temp->next;
}
```

Answer: This prints every number in the list from start to finish.

TOPIC 3: STACK (LIFO)

(Likelihood in Exam: 100% - Diagrams and Calculations)

The Concept: Last-In, First-Out (LIFO)

Think of a stack of heavy plates.

- You add to the **Top**. (This operation is called **PUSH**).
- You remove from the **Top**. (This operation is called **POP**).
- The first plate you put down is the **last** one you can pick up.

Typical Exam Question 1: "Show the Stack Content"

You will be given a list of operations:

1. Push(A) -> Stack: [A]
2. Push(B) -> Stack: [A, B] (B is on top)
3. Pop() -> Stack: [A] (B is removed)
4. Push(C) -> Stack: [A, C]
   Exam Tip: Draw a vertical box. Add items from the bottom up. Cross them out when Popped.

Typical Exam Question 2: Infix to Postfix Conversion

This appears in **every single final**.

- **Infix:** A + B (Normal math).
- **Postfix:** A B + (Computer math).

The Magic Method to Solve This (Manually):
Rules of Strength (Precedence):

1. * and / are **Strong**.
2. + and - are **Weak**.

Step-by-Step Algorithm (Memorize for Section B):
Example: A + B * C

1. Write A.
2. See +. It is an operator. Put it in the **Stack**.
3. Write B.
4. See *. Compare * with + (inside stack).
   - Is * stronger than +? Yes.
   - **Push** * on top of +.
5.
6. Write C.
7. End of line? Pop everything from stack.
   - Pop *, then Pop +.
     Result: A B C * +
8.

Parentheses Rule ():
If you see (, push it. If you see), pop everything until you hit (.

Typical Exam Question 3: Evaluating Postfix

Question: Calculate 5 3 + 2 *
Method:

1. Scan Left to Right.
2. If Number: **Push** to Stack.
    - Stack: [5, 3]
3.
4. If Operator (+): **Pop** two numbers.
    - Pop 3, Pop 5.
    - Math: 5 + 3 = 8.
    - **Push** 8. -> Stack: [8]
5.
6. If Number (2): **Push**.
    - Stack: [8, 2]
7.
8. If Operator (*): **Pop** two numbers.
    - Pop 2, Pop 8.
    - Math: 8 * 2 = 16.
    - **Push** 16.
9.
10. **Result:** 16.

TOPIC 4: QUEUE (FIFO)

(Likelihood in Exam: 100% - Array vs Linked List Comparison)

The Concept: First-In, First-Out (FIFO)

Think of a line at McDonald's.

- **Front:** People leave from here (Service).
- **Rear:** People arrive here (Wait).
- **Operations:**
    - EnQueue: Add to Rear.
    - DeQueue: Remove from Front.
-

Exam Problem 1: Linear Array Drawback (Rightward Drift)

Question: "What is the problem with Linear Array Queue?"

- **Answer:** Even if the queue is empty, the pointers (Front and Rear) keep moving to the right. Eventually, Rear hits the end of the array, and the computer thinks it is "Full" even though there is empty space at the beginning.
- *Solution:* Use a **Circular Queue**.

Exam Problem 2: Circular Array Calculation

Question: How do you move the Rear pointer in a circle?
You cannot just do Rear + 1.
Magic Formula:
Rear = (Rear + 1) % Size
This formula wraps the number back to 0 when it reaches the end.

Exam Problem 3: Linked List Queue Code

(This appeared in the 2018 Final Exam).

Insert (EnQueue):

- You use Rear->next to link the new node.
- You move Rear to the new node.

Remove (DeQueue):

- You verify queue is not empty (!isEmpty).
- You move Front to Front->next.
- You delete the old front node.

TOPIC 5: TREES & BST

(Likelihood in Exam: 100% - Visuals & Traversals)

Binary Tree (The General Type)

A parent has max 2 children (Left and Right).

- **Full Tree:** Every node has 0 or 2 children. (No single child).
- **Complete Tree:** Every level is full, filled from Left to Right.

Binary Search Tree (BST) - VERY IMPORTANT

The Math Rule:

- Left Child < Parent
- Right Child > Parent

Exam Task: Draw a BST.
Input: 10, 5, 15, 3.

1. 10 is Root.
2. 5: Smaller than 10? Yes. Go Left. Place 5.
3. 15: Bigger than 10? Yes. Go Right. Place 15.
4. 3: Smaller than 10? Left. Smaller than 5? Left. Place 3.

Resulting Tree:

code Code
downloadcontent_copy
expand_less

```
10
   / \
  5   15
 /
3
```

Exam Task: Traversals (Visiting Orders)

Given a tree, write the order.

Pre-Order: Root
→\rightarrow→

Left
→\rightarrow→

- Right
  (Example from above: 10, 5, 3, 15)

In-Order: Left
→\rightarrow→

Root
→\rightarrow→

- Right
  Magic Tip: In a Binary Search Tree, In-Order traversal **ALWAYS** results in numbers sorted from small to big (3, 5, 10, 15).

Post-Order: Left
→\rightarrow→

Right
→\rightarrow→

- **Root**
  (Example from above: 3, 5, 15, 10)

Exam Task: Deletion in BST

- **Case 1 (Leaf):** Easy. Just delete it.
- **Case 2 (One Child):** Easy. The parent adopts the grandchild.
- **Case 3 (Two Children): HARD.**
  - Find the node to delete (e.g., Node N).
  - Go to the **Right Subtree**. Find the **Smallest** number there (this is the leftmost node in the right side).
  - Replace N with that smallest number.
-

SECTION 3: MAGIC CHEAT SHEET FOR THE EXAM

If you memorize nothing else, **memorize this list**:

Part A: Comparison of Structures

Structure	Access Time	Search Time	Insertion/Deletion	Memory
Array	O(1) Fast	O(n) or O(log n)	Slow (need shifting)	Fixed Size
Linked List	O(n) Slow	O(n)	Fast (pointer change)	Dynamic (Pointers use extra space)
Stack	O(1) (Top only)	N/A	Fast (Push/Pop)	Restricted
Queue	O(1) (Front/Back)	N/A	Fast (Enq/Deq)	Restricted

Part B: Magic Code Lines to Write

Linked List Traversal:

code C++
downloadcontent_copy
expand_less
```
Node *temp = head;
while (temp != NULL) {
    cout << temp->data << " ";
    temp = temp->next;
}
```

New Node Creation:

code C++
downloadcontent_copy
expand_less
```
Node *newNode = new Node;
newNode->data = 10;
newNode->next = NULL;
```

Queue (Circular) Wrap Formula:

code C++
downloadcontent_copy

expand_less
back = (back + 1) % max_size;

Stack Push (Array):

code C++
downloadcontent_copy
expand_less
top++;
array[top] = data;

Tree Node Struct:

code C++
downloadcontent_copy
expand_less
struct TreeNode {
    int info;
    TreeNode *left;
    TreeNode *right;
};

Final Advice for the Exam Day

1. **For Tracing Questions:** Draw it out on paper. Draw the box. Draw the arrow. Do not do it in your head.
2. **For Coding Questions:** Even if you forget exact C++ syntax (semicolons, brackets), write the **Logic** clearly (e.g., temp = temp->next). Teachers give marks for logic.
3. **For Traversal:**
    - Draw an outline around the tree.
    - If your finger touches the **left** of the node first -> **PreOrder**.
    - If your finger touches the **bottom** -> **InOrder**.
    - If your finger touches the **right** -> **PostOrder**.
4.
5. **Binary Search Math:** remember (L+R)/2. 7/2 = 3 (in computer integer math, drop the 0.5).

You now have the knowledge contained in 10 files condensed into one. Good luck!